PRACTICAL 3

3. Implement routines to normalize the data in the sample data file using:

a) Min-Max normalization

(i) Map to range [0,1]

(ii) Map to range [-1,1]

b) z-score normalization

c) decimal scaled normalization

Sample DataSet : 10, 12, 3, 6, 5, 25, 17, 100, 1000, 98, 11, 27, 78, 33, 9, 18, 23, 44, 690, 200

PROGRAM

```
#include<stdio.h>
#include<math.h>

void min_max(int n,float data[],float ans[]);
void z_score(int n,float data[],float ans[]);
void decimal_scaling(int n,float data[],float ans[]);

int main(){

    int i,n,ch;
    float data[33],ans[33];
    printf("PRACTICAL 2: NORMALIZATION\n");
    printf("How many dataset value you want to enter: ");
    scanf("%d",&n);
    printf("Enter the dataset value: \n");
    for(i=0;i<n;i++){
            scanf("%f",&data[i]);
    }

    printf("Enter :\n1. Min-Max normalization\n2. Z-score
normalization\n3. decimal scaled normalization\n4. Exit\n");
```

```
    printf("\nEnter your Choice: ");
    scanf("%d",&ch);

    while(ch!=4){
        switch(ch){
            case 1: printf("\n");
                    min_max(n,data,ans);
            break;
            case 2: printf("\n");
                    z_score(n,data,ans);
            break;
            case 3: printf("\n");
                    decimal_scaling(n,data,ans);
            break;
            case 4: printf("Exiting the Program");
            break;
            default:printf("Enter the Choice Between 0-4\n");
        }
        printf("Enter your next choice: ");
        scanf("%d",&ch);
    }
return 0;
}

void min_max(int n,float data[],float ans[]){

    int i;
    float min,max,nmin,nmax,sub,div,mul;

    printf("Min-Max normalization\n");
    min=data[0];
    max=data[0];
    for(i=1;i<n;i++){
        if(min>data[i]){
            min=data[i];
        }
```

```
            if(max<data[i]){
                max=data[i];
            }
        }

        printf("Enter the range values: \n");
        scanf("%f %f",&nmin,&nmax);

        sub=nmax-nmin;

        for(i=0;i<n;i++){
            div=(data[i]-min)/(max-min);
            mul=div*sub;
            ans[i]=mul+nmin;
        }

        printf("\nThe Output is :\n");
        for(i=0;i<n;i++){
            printf("%f\n",ans[i]);
        }
    }

    void z_score(int n,float data[],float ans[]){

        int i;
        float mean=0.0,sum=0.0,s=0.0,sub=0.0,sd=0.0;

        printf("z-score normalization\n");
        for(i=0;i<n;i++){
            sum+=data[i];
        }
        mean=sum/n;

        printf("The mean values: %f \n",mean);
        for(i=0;i<n;i++){
            s+=pow((data[i]-mean),2);
```

```
    }
    sub=s/(n-1);
    sd=sqrt(sub);
    printf("The standard deviation values: %f \n",sd);

    for(i=0;i<n;i++){
        ans[i]=((data[i]-mean)/sd);
    }
    printf("\nThe Output is :\n");
    for(i=0;i<n;i++){
        printf("%f\n",ans[i]);
    }
}

void decimal_scaling(int n,float data[],float ans[]){

    int i,j=0,max;

    printf("Decimal Scaling\n");
    max=data[0];
    for(i=1;i<n;i++){
        if(max<data[i]){
            max=data[i];
        }
    }

    while(max>0){
        max/=10;
        j++;
    }

    for(i=0;i<n;i++){
        ans[i]=(data[i]/pow(10,j));
    }

    printf("\nThe Output is :\n");
```

```
        for(i=0;i<n;i++){
                printf("%f\n",ans[i]);
        }
    }
```

OUTPUT

(1) Min-Max normalization

(i) Map to range [0,1]

```
PRACTICAL 3: NORMALIZATION
How many dataset value you want to enter: 20
Enter the dataset value:
10
12
3
6
5
25
17
100
1000
98
11
27
78
33
9
18
23
44
690
200
Enter :
1. Min-Max normalization
2. Z-score normalization
3. decimal scaled normalization
4. Exit

Enter your Choice: 1
```

Min-Max normalization
Enter the range values:
0
1

The Output is :
0.007021
0.009027
0.000000
0.003009
0.002006
0.022066
0.014042
0.097292
1.000000
0.095286
0.008024
0.024072
0.075226
0.030090
0.006018
0.015045
0.020060
0.041123
0.689067
0.197593

(ii) Map to range [-1,1]

Enter your next choice: 1

Min-Max normalization
Enter the range values:
-1
1

The Output is :
-0.985958
-0.981946
-1.000000
-0.993982
-0.995988

-0.955868
-0.971916
-0.805416
1.000000
-0.809428
-0.983952
-0.951856
-0.849549
-0.939819
-0.987964
-0.969910
-0.959880
-0.917753
0.378134
-0.604814

(2) Z-Score normalization

Enter your Choice: 2

z-score normalization
The mean values: 120.449997
The standard deviation values: 257.279388

The Output is :
-0.429300
-0.421526
-0.456508
-0.444847
-0.448734
-0.370997
-0.402092
-0.079486
3.418657
-0.087259
-0.425413
-0.363224
-0.164996
-0.339903
-0.433187
-0.398205
-0.378771
-0.297148

2.213741
0.309197

(3) Decimal Scaled normalization

Enter your next choice: 3

Decimal Scaling

The Output is :
0.001000
0.001200
0.000300
0.000600
0.000500
0.002500
0.001700
0.010000
0.100000
0.009800
0.001100
0.002700
0.007800
0.003300
0.000900
0.001800
0.002300
0.004400
0.069000
0.020000

PRACTICAL 4

4. Implement Binning methods for data smoothing for the following dataset using 3 equi-depth bins. Demonstrate
(a) smoothing by bin means,
(b) smoothing by bin medians
(c) bin boundaries.

Sample Dataset: 13, 15, 16, 16, 19, 20, 20, 21, 22, 22, 25, 25, 25, 25, 30, 33, 33, 35, 35, 35, 35, 36, 40, 45, 46, 52, 70

PROGRAM

```
#include<stdio.h>
#include<math.h>
#include<conio.h>

void sort(int data[],int n);
void mean(int bin_size,int depth,int bucket[][11]);
void median(int bin_size,int depth,int bucket[][11]);
void boundary(int bin_size,int depth,int bucket[][11]);

int main(){

   int n,i,j,bin_size,a=0,data[33],bucket[11][11],ch,depth;

   printf("PRACTICAL 4: BINNING METHODS FOR DATA SMOOTHING\n\n");
   printf("How many dataset value you want to enter: ");
   scanf("%d",&n);
   printf("Enter the dataset value: \n");
   for(i=0;i<n;i++){
      scanf("%d",&data[i]);
   }
   printf("Enter the Depth value: ");
   scanf("%d",&depth);
```

```
   printf("\nSorted data is: \n");
   sort(data,n);
   for(i=0;i<n;i++){
   printf("%d\n",data[i]);
   }

   bin_size=n/depth;  //No. of buckets we use
   for(i=0;i<bin_size;i++){
      for(j=0;j<depth;j++){
         bucket[i][j]=data[a];
         a++;
      }
   }

   for(i=0;i<bin_size;i++){
      printf("\nBucket/Bin%d:",i+1);
      for(j=0;j<depth;j++){
         printf(" %3d   ",bucket[i][j]);
      }
   }

   printf("\nEnter :\n1. Smoothing by bin means\n2. Smoothing by bin
medians\n3. Smoothing by Bin Boundaries\n4. Exit\n");
   printf("\nEnter your Choice: ");
   scanf("%d",&ch);

   while(ch!=4){
      switch(ch){
         case 1: printf("\nSmoothing by bin means\n");
               mean(bin_size,depth,bucket);
               printf("\n");
         break;
         case 2: printf("\nSmoothing by bin medians\n");
               median(bin_size,depth,bucket);
               printf("\n");
```

```
        break;
        case 3: printf("\nSmoothing by Bin Boundaries\n");
            boundary(bin_size,depth,bucket);
            printf("\n");
        break;
        case 4: printf("Exiting the Program");
        break;
        default:printf("Enter the Choice Between 0-4\n");
      }
      printf("Enter your next choice: ");
      scanf("%d",&ch);
   }

   return 0;

}

void sort(int data[],int n){

   int i,j,temp=0;
   for(i=0;i<n;i++){
      for(j=i+1;j<n;j++){
         if(data[i]>data[j]){
            temp=data[i];
            data[i]=data[j];
            data[j]=temp;
         }
      }
   }
   return;
}

void mean(int bin_size,int depth,int bucket[][11]){
   int i,j,sum=0,mean=0,bin[11][depth];

   for(i=0;i<bin_size;i++){
```

```
        sum=0;
        for(j=0;j<depth;j++){
            sum+=bucket[i][j];
        }
        mean=sum/depth;
        for(j=0;j<depth;j++){
            bin[i][j]=mean;
        }
    }

    for(i=0;i<bin_size;i++){
        printf("\nBucket/Bin%d:",i+1);
        for(j=0;j<depth;j++){
            printf(" %3d   ",bin[i][j]);
        }
    }
    printf("\n");
    return;
}

void median(int bin_size,int depth,int bucket[][11]){
    int i,j,bin[11][depth],a,b,median=0;

    for(i=0;i<bin_size;i++){
        if(depth % 2 == 0){
            a=depth/2;
            b=(depth/2)+1;

            median=(bucket[i][a-1]+bucket[i][b-1])/2;
        }
        else{
            a=(depth+1)/2;
            median=bucket[i][a-1];
        }

        for(j=0;j<depth;j++){
```

```
            bin[i][j]=median;
        }
    }
    for(i=0;i<bin_size;i++){
        printf("\nBucket/Bin%d:",i+1);
        for(j=0;j<depth;j++){
            printf(" %3d   ",bin[i][j]);
        }
    }
    printf("\n");
    return;
}

void boundary(int bin_size,int depth,int bucket[][11]){
    int i,j,bin[11][depth],a,b,min,max;

    for(i=0;i<bin_size;i++){
        min=bucket[i][0];
        max=bucket[i][depth-1];

        for(j=0;j<depth;j++){
            a=bucket[i][j]-min;
            b=max-bucket[i][j];

            if(a<=b){
                bin[i][j]=min;
            }
            else{
                bin[i][j]=max;
            }
        }
    }
    for(i=0;i<bin_size;i++){
        printf("\nBucket/Bin%d:",i+1);
        for(j=0;j<depth;j++){
            printf(" %3d   ",bin[i][j]);
```

```
      }
    }
    printf("\n");
    return;
}
```

OUTPUT

(1) Smoothing By Bin Means

```
PRACTICAL 4: BINNING METHODS FOR DATA SMOOTHING
How many dataset value you want to enter: 27
Enter the dataset value:
13
15
16
16
19
20
20
21
22
22
25
25
25
25
30
33
33
35
35
35
35
36
40
45
46
52
70
Enter the Depth value: 3
Sorted data is:
13
```

15
16
16
19
20
20
21
22
22
25
25
25
25
30
33
33
35
35
35
35
36
40
45
46
52
70

Bucket/Bin1: 13 15 16
Bucket/Bin2: 16 19 20
Bucket/Bin3: 20 21 22
Bucket/Bin4: 22 25 25
Bucket/Bin5: 25 25 30
Bucket/Bin6: 33 33 35
Bucket/Bin7: 35 35 35
Bucket/Bin8: 36 40 45
Bucket/Bin9: 46 52 70
Enter :
1. Smoothing by bin means
2. Smoothing by bin medians
3. Smoothing by Bin Boundaries
4. Exit

```
Enter your Choice: 1
Smoothing by bin means
Bucket/Bin1:  14    14    14
Bucket/Bin2:  18    18    18
Bucket/Bin3:  21    21    21
Bucket/Bin4:  24    24    24
Bucket/Bin5:  26    26    26
Bucket/Bin6:  33    33    33
Bucket/Bin7:  35    35    35
Bucket/Bin8:  40    40    40
Bucket/Bin9:  56    56    56
```

(2) Smoothing By Bin Medians

```
Enter your next choice: 2
Smoothing by bin medians
Bucket/Bin1:  15    15    15
Bucket/Bin2:  19    19    19
Bucket/Bin3:  21    21    21
Bucket/Bin4:  25    25    25
Bucket/Bin5:  25    25    25
Bucket/Bin6:  33    33    33
Bucket/Bin7:  35    35    35
Bucket/Bin8:  40    40    40
Bucket/Bin9:  52    52    52
```

(3) Smoothing By Bin Boundaries

```
Enter your next choice: 3
Smoothing by Bin Boundaries
Bucket/Bin1:  13    16    16
Bucket/Bin2:  16    20    20
Bucket/Bin3:  20    20    22
Bucket/Bin4:  22    25    25
Bucket/Bin5:  25    25    30
Bucket/Bin6:  33    33    35
Bucket/Bin7:  35    35    35
Bucket/Bin8:  36    36    45
Bucket/Bin9:  46    46    70

Enter your next choice: 4
PS D:\DMBI>
```

PRACTICAL 5

5. Generate a linear regression based model and demonstrate its working on the given dataset.

PROGRAM

```
#include<stdio.h>
#include<math.h>

void li_reg(float x[],float y[],int n,float w0,float w1);
int main(){

    int i,n;
    float x[33],y[33];
    float w0,w1;
    printf("PRACTICAL 5: LINEAR REGRESSION\n");
    printf("How many dataset value you want to enter: ");
    scanf("%d",&n);
    printf("Enter the value of X: \n");
    for(i=0;i<n;i++){
        scanf("%f",&x[i]);
    }
    printf("Enter the value of Y: \n");
    for(i=0;i<n;i++){
        scanf("%f",&y[i]);
    }
    li_reg(x,y,n,w0,w1);
    return 0;
}
void li_reg(float x[],float y[],int n,float w0,float w1){
    int i;
    float meanx,meany,sum=0.0,sum1=0.0,check,ans,a[33],b[33],c[33],d[33]; //a=x-
meanx b=y-meany c=(x-meanx)(y=meany) d=(x-meanx)^2 check for taking x ans
for calculating y
    for(i=0;i<n;i++){
        sum+=x[i];
    }
    meanx=sum/n;
    sum=0.0;
    for(i=0;i<n;i++){
```

```
            sum+=y[i];
    }
    meany=sum/n;

    sum=0.0;
    for(i=0;i<n;i++){
            a[i]=x[i]-meanx;
    }
    for(i=0;i<n;i++){
            b[i]=y[i]-meany;
    }
    for(i=0;i<n;i++){
            c[i]=a[i]*b[i];
            sum+=c[i];
    }
    for(i=0;i<n;i++){
            d[i]=pow(a[i],2);
            sum1+=d[i];
    }
    printf("The table is :\n");
    for(i=0;i<n;i++){
            printf("%11f %11f %11f %11f %11f %11f ",x[i],y[i],a[i],b[i],c[i],d[i]);
            printf("\n");
    }
    w1=sum/sum1;
    w0=meany-(w1*meanx);
    printf("\nW0 : %f\n",w0);
    printf("W1 : %f\n",w1);

    printf("The Linear Regression formula:   ");
    printf("y = %f + %f x",w0,w1);

    printf("\n\nEnter the Value of x To be checked:\n");
    scanf("%f",&check);
    ans=w0 + (w1 * check);
    printf("Y = %f",ans);
}
```

OUTPUT

```
PRACTICAL 5: LINEAR REGRESSION
How many dataset value you want to enter: 10
Enter the value of X:
```

```
3
8
9
13
3
6
11
21
1
16
Enter the value of Y:
30
57
64
72
36
43
59
90
20
83
The table is :
   3.000000   30.000000   -6.100000  -25.400002  154.940018   37.210003
   8.000000   57.000000   -1.100000    1.599998   -1.759999    1.210001
   9.000000   64.000000   -0.100000    8.599998   -0.860003    0.010000
  13.000000   72.000000    3.900000   16.599998   64.739990   15.209997
   3.000000   36.000000   -6.100000  -19.400002  118.34001   37.210003
   6.000000   43.000000   -3.100000  -12.400002   38.440010    9.610003
  11.000000   59.000000    1.900000    3.599998    6.839996    3.609998
  21.000000   90.000000   11.900000   34.59999   411.739960  141.609985
   1.000000   20.000000   -8.100000  -35.400002  286.740021   65.610008
  16.000000   83.000000    6.900000   27.599998  190.439972   47.609993

W0 : 23.208973
W1 : 3.537476
The Linear Regression formula:   y = 23.208973 + 3.537476 x

Enter the Value of x To be checked:
10
Y = 58.583729
PS D:\DMBI>
```

PRACTICAL 11

11. Find the Manhattan, Euclidean and Minkowski Distance Between two input data points, Data Points could be Single Dimension or Two Dimensional or Three Dimensional points.

PROGRAM

```
#include<stdio.h>
#include<stdlib.h>
#include<math.h>

int main(){
	int dim,i,d1[3],d2[3],res[3],sum=0,p;
	double result=0;

	printf("IMPLEMENTATION OF \n1. MANHATTAN DISTANCE\n2.
EUCLIDEAN DISTANCE\n3. MINKOWSKI DISTANCE\n\n");
	printf("Enter the dimension of Dataset: ");
	scanf("%d",&dim);

	printf("Enter 1st points:\n");
	for(i=0;i<dim;i++){
		scanf("%d",&d1[i]);
	}

	printf("Enter 2nd points:\n");
	for(i=0;i<dim;i++){
		scanf("%d",&d2[i]);
	}
	printf("\nPoints are: \n");
	printf("(");
	for(i=0;i<dim;i++){
		printf("%d  ",d1[i]);
	}
	printf(")\n(");
	for(i=0;i<dim;i++){
```

```
            printf("%d  ",d2[i]);
        }
        printf(")\n");

        printf("\nMANHATTAN DIATANCE:\n");
        for(i=0;i<dim;i++){
            res[i]= abs((d1[i]-d2[i]));
            sum+=res[i];
        }

        printf("Manhattan Distance is: %d\n",sum);

        printf("\nEUCLIDEAN DIATANCE:\n");
        sum=0;
        for(i=0;i<dim;i++){
            res[i]= pow((d1[i]-d2[i]),2);
            sum+=res[i];
        }
        result= sqrt(sum);
        printf("Distance is: %d\n",sum);
        printf("Euclidean Distance is: %lf\n",result);

        printf("\nMINKOWSKI DIATANCE:\n");
        printf("Enter p value: ");
        scanf("%d",&p);
        sum=0;
        for(i=0;i<dim;i++){
            res[i]= pow((d1[i]-d2[i]),p);
            sum+=res[i];
        }
        result=0;
        result= pow(sum,(1./p));
        printf("Distance is: %d\n",sum);
        printf("Euclidean Distance is: %lf\n",result);
        return 0;
}
```

OUTPUT

```
IMPLEMENTATION OF
1. MANHATTAN DISTANCE
2. EUCLIDEAN DISTANCE
3. MINKOWSKI DISTANCE

Enter the dimension of Dataset: 2
Enter 1st points:
5
10
Enter 2nd points:
1
2

Points are:
(5  10  )
(1  2  )

MANHATTAN DISTANCE:
Manhattan Distance is: 12

EUCLIDEAN DISTANCE:
Distance is: 80
Euclidean Distance is: 8.944272

MINKOWSKI DISTANCE:
Enter p value: 5
Distance is: 33792
Euclidean Distance is: 8.049386
PS D:\DMBI>
```

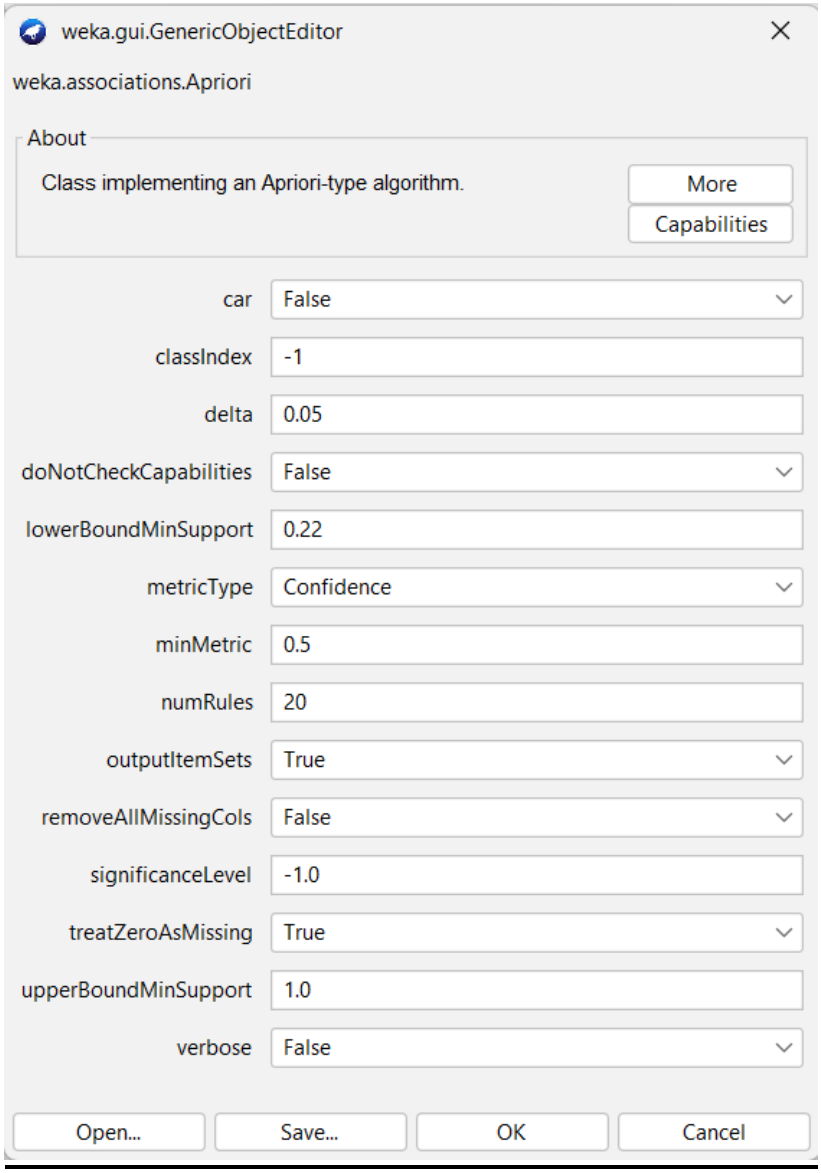
PRACTICAL 6

6. Study and implement the Apriori algorithm over the given dataset. (Use WEKA toolkit).

ATTRIBUTE - RELATION FILE FORMAT (Data file used)

```
@RELATION basket5
@ATTRIBUTE i1{0,1}
@ATTRIBUTE i2{0,1}
@ATTRIBUTE i3{0,1}
@ATTRIBUTE i4{0,1}
@ATTRIBUTE i5{0,1}
@DATA
1,1,0,0,1
0,1,0,1,0
0,1,1,0,0
1,1,0,1,0
1,0,1,0,0
0,1,1,0,0
1,0,1,0,0
1,1,1,0,1
1,1,1,0,0
```

ASSOCIATOR



ASSOCIATOR OUTPUT

=== Run information ===

Scheme: weka.associations.Apriori -I -N 20 -T 0 -C 0.5 -D 0.05 -U 1.0 -M 0.22 -S -1.0 -Z -c -1
Relation: basket5
Instances: 9
Attributes: 5

```
        i1
        i2
        i3
        i4
        i5
=== Associator model (full training set) ===

Apriori
=======

Minimum support: 0.22 (2 instances)
Minimum metric <confidence>: 0.5
Number of cycles performed: 16

Generated sets of large itemsets:

Size of set of large itemsets L(1): 5

Large Itemsets L(1):
i1=1    6
i2=1    7
i3=1    6
i4=1    2
i5=1    2

Size of set of large itemsets L(2): 6

Large Itemsets L(2):
i1=1 i2=1      4
i1=1 i3=1      4
i1=1 i5=1      2
i2=1 i3=1      4
i2=1 i4=1      2
i2=1 i5=1      2
```

Size of set of large itemsets L(3): 2

Large Itemsets L(3):
i1=1 i2=1 i3=1 2
i1=1 i2=1 i5=1 2

Best rules found:

1. i5=1 2 ==> i1=1 2 <conf:(1)> lift:(1.5) lev:(0.07) [0] conv:(0.67)
2. i4=1 2 ==> i2=1 2 <conf:(1)> lift:(1.29) lev:(0.05) [0] conv:(0.44)
3. i5=1 2 ==> i2=1 2 <conf:(1)> lift:(1.29) lev:(0.05) [0] conv:(0.44)
4. i2=1 i5=1 2 ==> i1=1 2 <conf:(1)> lift:(1.5) lev:(0.07) [0] conv:(0.67)
5. i1=1 i5=1 2 ==> i2=1 2 <conf:(1)> lift:(1.29) lev:(0.05) [0] conv:(0.44)
6. i5=1 2 ==> i1=1 i2=1 2 <conf:(1)> lift:(2.25) lev:(0.12) [1] conv:(1.11)
7. i1=1 6 ==> i2=1 4 <conf:(0.67)> lift:(0.86) lev:(-0.07) [0] conv:(0.44)
8. i3=1 6 ==> i1=1 4 <conf:(0.67)> lift:(1) lev:(0) [0] conv:(0.67)
9. i1=1 6 ==> i3=1 4 <conf:(0.67)> lift:(1) lev:(0) [0] conv:(0.67)
10. i3=1 6 ==> i2=1 4 <conf:(0.67)> lift:(0.86) lev:(-0.07) [0] conv:(0.44)
11. i2=1 7 ==> i1=1 4 <conf:(0.57)> lift:(0.86) lev:(-0.07) [0] conv:(0.58)
12. i2=1 7 ==> i3=1 4 <conf:(0.57)> lift:(0.86) lev:(-0.07) [0] conv:(0.58)
13. i2=1 i3=1 4 ==> i1=1 2 <conf:(0.5)> lift:(0.75) lev:(-0.07) [0] conv:(0.44)
14. i1=1 i3=1 4 ==> i2=1 2 <conf:(0.5)> lift:(0.64) lev:(-0.12) [-1] conv:(0.3)
15. i1=1 i2=1 4 ==> i3=1 2 <conf:(0.5)> lift:(0.75) lev:(-0.07) [0] conv:(0.44)
16. i1=1 i2=1 4 ==> i5=1 2 <conf:(0.5)> lift:(2.25) lev:(0.12) [1] conv:(1.04)

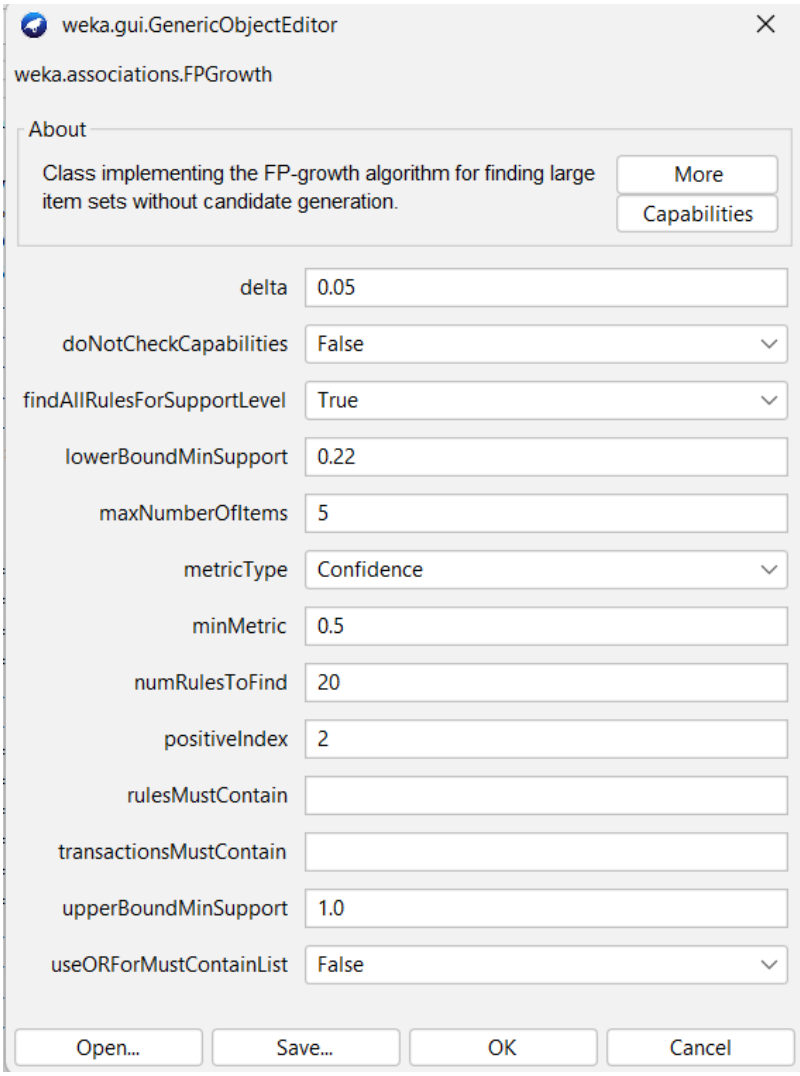
PRACTICAL 7

7. Study and implement the FP-Growth algorithm over the given dataset. (Use WEKA toolkit).

ATTRIBUTE - RELATION FILE FORMAT (DataSet used)

```
@RELATION basket5
@ATTRIBUTE i1{0,1}
@ATTRIBUTE i2{0,1}
@ATTRIBUTE i3{0,1}
@ATTRIBUTE i4{0,1}
@ATTRIBUTE i5{0,1}
@DATA
1,1,0,0,1
0,1,0,1,0
0,1,1,0,0
1,1,0,1,0
1,0,1,0,0
0,1,1,0,0
1,0,1,0,0
1,1,1,0,1
1,1,1,0,0
```

ASSOCIATOR



ASSOCIATOR OUTPUT

=== Run information ===

Scheme: weka.associations.FPGrowth -P 2 -I 5 -N 20 -T 0 -C 0.5 -D 0.05 -U 1.0 -M 0.22 -S
Relation: basket5
Instances: 9
Attributes: 5
i1
i2
i3

```
        i4
        i5
=== Associator model (full training set) ===

FPGrowth found 16 rules

 1. [i5=1]: 2 ==> [i2=1]: 2   <conf:(1)> lift:(1.29) lev:(0.05) conv:(0.44)
 2. [i4=1]: 2 ==> [i2=1]: 2   <conf:(1)> lift:(1.29) lev:(0.05) conv:(0.44)
 3. [i5=1]: 2 ==> [i1=1]: 2   <conf:(1)> lift:(1.5) lev:(0.07) conv:(0.67)
 4. [i5=1]: 2 ==> [i2=1, i1=1]: 2   <conf:(1)> lift:(2.25) lev:(0.12) conv:(1.11)
 5. [i2=1, i5=1]: 2 ==> [i1=1]: 2   <conf:(1)> lift:(1.5) lev:(0.07) conv:(0.67)
 6. [i1=1, i5=1]: 2 ==> [i2=1]: 2   <conf:(1)> lift:(1.29) lev:(0.05) conv:(0.44)
 7. [i3=1]: 6 ==> [i2=1]: 4   <conf:(0.67)> lift:(0.86) lev:(-0.07) conv:(0.44)
 8. [i1=1]: 6 ==> [i2=1]: 4   <conf:(0.67)> lift:(0.86) lev:(-0.07) conv:(0.44)
 9. [i3=1]: 6 ==> [i1=1]: 4   <conf:(0.67)> lift:(1) lev:(0) conv:(0.67)
10. [i1=1]: 6 ==> [i3=1]: 4   <conf:(0.67)> lift:(1) lev:(0) conv:(0.67)
11. [i2=1]: 7 ==> [i3=1]: 4   <conf:(0.57)> lift:(0.86) lev:(-0.07) conv:(0.58)
12. [i2=1]: 7 ==> [i1=1]: 4   <conf:(0.57)> lift:(0.86) lev:(-0.07) conv:(0.58)
13. [i2=1, i3=1]: 4 ==> [i1=1]: 2   <conf:(0.5)> lift:(0.75) lev:(-0.07)
conv:(0.44)
14. [i2=1, i1=1]: 4 ==> [i3=1]: 2   <conf:(0.5)> lift:(0.75) lev:(-0.07)
conv:(0.44)
15. [i3=1, i1=1]: 4 ==> [i2=1]: 2   <conf:(0.5)> lift:(0.64) lev:(-0.12) conv:(0.3)
16. [i2=1, i1=1]: 4 ==> [i5=1]: 2   <conf:(0.5)> lift:(2.25) lev:(0.12) conv:(1.04)
```